Day 1 - Recap.

→ Basic concepts - "Generative AI" (FM / LLMs)
→ Amazon Bedrock & AI/ML stack by AWS
→ Amazon Bedrock & Amazon Bedrock "Agentcore"
↓ Development
↓ Deployment & upscale.
→ Amazon Bedrock - Model Catalog
Supported Model - Bedrock
& Pricing

→ Inference → Request → Model
Response ←
Control the behavior

→ Inference Parameters / Configuration ✓
1. Temperature ✓
2. Top P
3. Top K
4. Token Counts → 512
5. Stop Sequences → ['}'] # {Human}
100 - 150

Day 2 - Agenda.

* 1 ✓ Programming with Amazon Bedrock (Introduce to APIs) → "Inference Profile" → (2 labs) ↳ 30-45 mins)
2 ✓ Prompt Engineering
* 3 ✓ Using Amazon Bedrock APIs in Common Architectures → Lab 1 - (15-20 mins)
* 4 ✓ Customizing Generative AI Responses with RAG ↳ Lab - 1 - (30-45 mins)

Bedrock APIs (Runtime)
1. InvokeModel ✓
2. Converse ✓
↳ Model ID = " ___ "

Model A	Model B.
Model ID	Model ID
Temperature	Temp
Top P	top P
MaxTokens	max count

Invoke API	Converse API
→ Model-specific code	→ unified, model-agnostic schema
→ Manage conversational state MANUALLY	→ conversational AI (API) ↳ Built in
→ Stateless	
→ Raw binary (JSON)	→ clean
→ Payload Higher limit (5 MB) for larger files	→ lower limit (3.75 mb)

Model A → larger Response → THINKING + Generating Response
↓
5 PDFs → [Generate Summary]

1. ✓ Converse Stream API → unified conversational - optimized

2. InvokeModelWithResponseStream ↳ low-level, model specific Interface.

API Abstraction & Schema:
* ✓ ConverseStream → single, standardized schema.
✓ InvokeModelWithRS → Payloads, response parsing unique to each specific Model

IAM Permissions.

PROMPT
- 1
- 2
- 3
- 4
- 5
↓
Model A ✓

Metrics ✓ ✓ ✓ ✓ ✓ ✓ } Model as a Judge ↓ Model B

Job
1. Invoke Model A
2. Generate Response
3. Model as a Judge → check Metrics
4. Store evaluation results for Model A

"Inference Profile" = Control HOW & WHERE Model (FM/LLM) Inference requests are processed.
↳ especially across regions
1. Cross-region Inference (Req → Response)
2. Load Balancing Across region
3. HA
4. Better latency
5. Simplified Application Configuration

Why? → AWS to route to the BEST region
[us.
global.
eu.

What exactly is an "Inference Profile"?
↓
logical wrapper around one / more deployment across AWS Regions.

→ Act as a global alias for Model Inference.

How it works? (Flow)
1. Application Calls - Inference Profile ID
2. AWS evaluates
   - Region Health
   - Model Capacity
   - Latency
3. Request is routed to optimal region
4. Response is returned.

Types of Inference Profiles
1. Cross-Region (System-defined)
   - Provided by AWS
   - Prefix = us.
     global.
     eu.
     Au.

2. Application Inference Profiles
   → Track usage
   → Apply cost Allocation Tags
   → Control Routes preferences.
   * They're useful in enterprise setups, where:
     1. Dev / Stage / Prod must be separated
     2. Cost Tracking
     3. governance is IMP.

3. When Should you use it?
   Global Application ——— Yes
   HA needed ——— Yes
   Prod system ——— Yes
   Simple Testing / demo ——— No

Amazon Bedrock Inference Modes
1. Batch
2. on-demand ✓
3. Provisioned ✓

Batch - Inference Mode
Latency → High (mins to HR)
Cost → More efficient for large jobs.
use cases: AI Reports
Predictive analytics
Offline processing
Large document